失敗からの立て直しを学ぶロボット

自分が失敗した場面で、プログラムが立て直した学習データから学び直す VLA

Physical AI 応用 1 講座 最終課題　／　omnicampus アカウント名: Fuji1192

動画: https://youtu.be/y7Wzc8WhawE

コード: https://github.com/fujikro0814-del/PAI-final-Fuji1192

1. 概要（動画 0:00〜）

本作のアイデアは、人が手本を示しにくい「失敗からの立て直し」を、わざと起こした失敗と、ロボット自身が起こした失敗から自動で学ばせることです。

産業用ロボットは、失敗が許されない工業製品を扱うため、検証の難しい学習した方策には任せず、環境や把持物を事前に定義して手先の軌道を決定論的に計画します。LLM を使う場合も上位の判断にとどまり、とある展示会の実演でも、LLM が品目を決めて深度カメラなどの観測で部品を配置する一方、部品ごとの把持位置は人が種類ごとに教示していました（ファナック、JIMTOF2026、2026 年 10 月）。一方、家庭用のロボットは置き場所も物も毎回違う中で家事をこなす必要があり、基盤モデルによる方策が欠かせません。ただ、そうした方策は掴み損ねや落下を避けきれず、そのたびに人が拾い直すのでは役に立ちません。失敗からの立て直しは人が手本を示しにくく、データを集めにくいものです。そこで本作は、シミュレーションでの自動生成と、ロボット自身の失敗からの学び直しで、人の遠隔操作なしに立て直しを学ばせました（シミュレーションの立方体で検証）。

シミュレーション上の Franka Panda が、日本語の指示「全部片付けて」に従い、机の上の立方体 3 個を箱に入れます。VLA（SmolVLA を追加学習）がカメラ画像から腕を動かし、LLM（Claude）が指示を手順に分け、落下や時間切れのときに手順を組み直します。

成功データだけで学習した VLA に、(1) わざと起こした失敗から立て直すデータを足し、(2) VLA 自身が失敗した場面から立て直すデータを足して学び直し、(3) LLM による手順の組み直しを加え、(4) 推定した箱の空きセルを配置位置として VLA に与える目標条件付けを加えました。学習にも調整にも使っていない同じ 200 通りの初期状態で、3 個とも箱に入った割合は、(1)(2) の立て直しの学習だけで 39.0% から 72.0% に上がり（LLM なし）、学習とは別に (3)(4) を重ねて 96.5% になりました（図 1、表 1）。LLM の改良は学習用の初期状態だけで確かめ、評価の初期状態では調整していません。(4) は、残った失敗を分類した後に足したものです（3.5 節）。比べる条件と検定の方法は、評価の前に決めて固定しました。

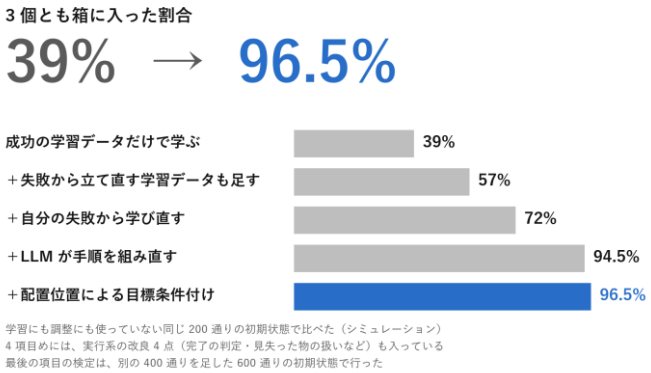


図 1　学び方ごとの、3 個とも箱に入った割合（同じ 200 通りの初期状態、シミュレーション）。

検定の細部・追加の数・再現の手順・参考文献の書誌は GitHub の README にまとめました。動きを滑らかにする手法 RTC を最終モデルに足した追加の試みは、付録 A にまとめました。

2. システム構成（動画 1:38〜）

全体を図 2 に示します。実行時に使うのは、実機でも得られるセンサの出力（RGB-D カメラ、関節の状態、指の幅）だけで、シミュレーションの真の状態は使いません。

- **方策（VLA）**: SmolVLA（Shukor ら, 2025）です。俯瞰と手首の RGB 画像 2 枚と状態（17 次元に配置位置の 2 次元を加えた 19 次元。3.5 節）から、手先の変位とグリッパの開閉を 50 ステップ分まとめて出します（行動チャンク（Zhao ら, 2023）、10 Hz、非同期実行）。推論にかかる時間は実測の分布で与え、その間もシミュレーションを止めずに前の行動列の続きを実行します（RTC を足したものは付録 A）。行動エキスパートと状態の射影だけを追加学習しました。
- **手順（LLM）**: Claude Haiku 5.5 が指示を色の手順に分けます。落下や同じ手順での時間切れの繰り返しのとき、Claude Sonnet 5.5 が状況を報告して手順を組み直します。LLM の出力は規則で検査し、通らなければ止まって人に知らせます。
- **低レベル制御**: 微分逆運動学（減衰つき最小二乗）で関節の指令に変え、位置・速度・加速度・躍度を制限します。
- **知覚・判定**: RGB-D 画像から物と箱の位置を推定し、完了と落下を判定します。

- **計算資源**: 学習・評価とも RTX 4080 SUPER ×1 です。初期値は LIBERO（Liu ら, 2023） で学習済みの SmolVLA、シミュレータは MuJoCo（Todorov ら, 2012）、学習は LeRobot（Cadene ら, 2024） を使いました。

3. 工夫した点

3.1 失敗から立て直す学習データの自動生成（動画 0:26〜）

生成プログラム（シミュレーションの正確な位置を使って動くスクリプト方策、学習なし）にデータを作らせ、その途中でわざと失敗させます。掴み損ねは掴む位置をずらすか閉じる高さを上げて起こし、置き損ねは机の上で放して起こします。失敗が物の状態で確定した時点から、生成プログラムが掴み直して箱に入れるまでを立て直しデータとして記録します（図 3a）。失敗を起こす前の部分は記録しないので、失敗の動作そのものは学習しません。成功データ 240 本と立て直しデータ 90 本で追加学習しました。

3.2 自分の失敗から学び直す（動画 0:35〜）

わざと起こした失敗は、VLA が実際に起こす失敗と分布が異なります。そこで、学習した VLA を学習用の初期状態で動かし、VLA が掴み損ねた・落とした・置く直前に外した直後に生成プログラムへ切り替え、その場から箱に入れるまでを記録しました（図 3b）。方策が実際に訪れる状態で教師のデータを集める点で、DAgger（Ross ら, 2011） と同じ考え方です。この交代後のデータ 200 本に、外乱を入れたデータ 80 本と、箱に 2 個入った場面の成功データ 60 本を加え、計 670 本で学び直しました。

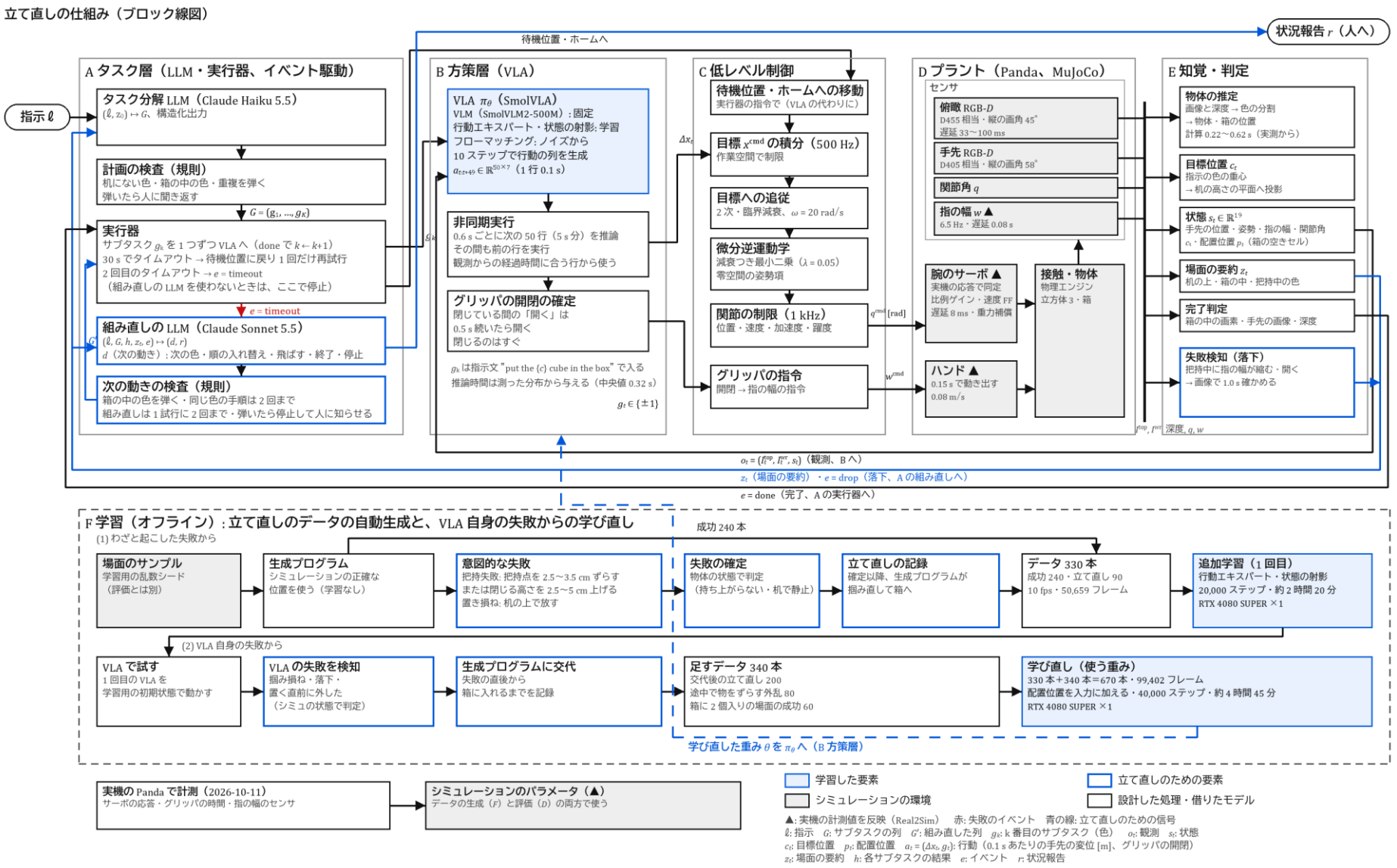


図 2　立て直しの仕組み（ブロック線図）。A〜E は実行時、F は学習（オフライン）。青の塗りは学習した要素、青の枠は立て直しのための要素、灰色はシミュレーションの環境（▲ は実機の計測値を反映）。記号の定義は図の右下。

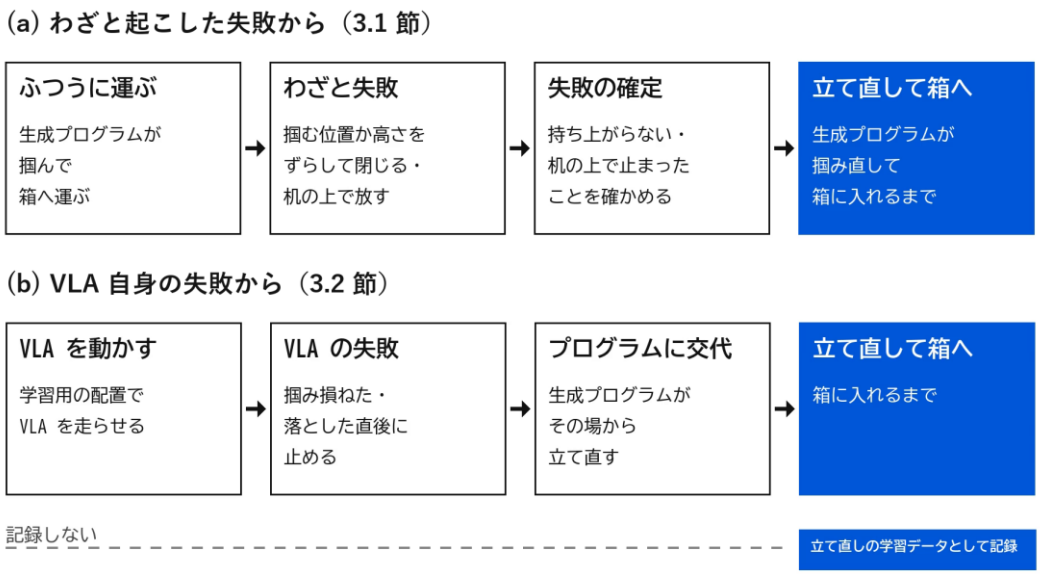


図 3　立て直しの学習データの作り方。(a) わざと起こした失敗から、(b) VLA 自身の失敗から。どちらも失敗より後の部分だけを記録する。

3.3 失敗の検知と LLM による手順の組み直し（動画 0:50〜）

方策だけでは抜け出せない失敗（運搬中の落下、同じ色での時間切れの繰り返し、腕に隠れた物の見失い）は、失敗の検知をきっかけに LLM が扱います。LLM は場面の要約から次の動きを選び、日本語 2 文以内で状況を報告します。報告の例（動画 1:03 頃）:「緑の立方体を運ぶ途中で落としてしまい、箱には今のところ赤だけが入っています。もう 1 度緑を試し、そのあと青を入れます。」

評価の前に、学習を伴わない実行系の改良を 4 点（LLM への指示文、場面の要約、完了判定、見失った物の扱い）加えました。いずれも学習用の初期状態だけで確かめました（中身は README）。

3.4 実機で測った値を反映したシミュレーション（動画 2:17〜）

実機の Panda（カメラなし）で生成プログラムの関節の指令を再生して腕とグリッパの応答を測り、シミュレーションのサーボ・遅延・グリッパの応答を合わせて、この条件でデータの生成と学習をやり直しました（腕の追従の遅延は約 100 ms から 6 ms、実機 6〜8 ms）。方策が出した腕の動きを実機で再生すると、手先の位置の差は 1 mm 未満でした（測定と再生の詳細は README）。学習した VLA を実機で閉じたループで動かす評価は、今後の課題です。

3.5 配置位置による目標条件付け（動画 1:26〜）

LLM の組み直しまで加えたものを、別の 400 通りを足した 600 通りの初期状態で評価し、失敗 30 本を分類しました。18 本は、収納済みの物体の上でグリッパを開いて物体を放し、物体がその上に載ったまま静止したもの（箱に入ったと判定しない）でした。VLA の入力には配置位置の情報がなく、放す位置がばらつくことが原因と考えました。そこで、VLA の状態入力に配置位置 $p \in \mathbb{R}^2$（箱内の水平位置）を加え、目標条件付き方策として学び直しました。学習では、各デモで対象物体が最終的に静止した位置を配置位置として事後的に付与し（hindsight relabeling）、データは 3.2 節の 670 本のまま

追加していません。実行時は、RGB-D から推定した箱の内寸を 2×2 のセルに分け、収納済みの物体から最も遠いセルの中心を配置位置として与えます（シミュレーションの真値は使いません）。配置位置はサブタスク（1 個ずつ）の開始時と再試行のたびに決め直し、その間は固定します。この比較は、失敗を分類した後・新しい条件を評価する前に決めて固定しました。

実際に到達した結果を事後的に目標とする考え方は Hindsight Experience Replay（Andrychowicz ら, 2017）と同じで、空いた配置位置を選んで与える点は RoboPoint（Yuan ら, 2024）に近いものです。RoboPoint は VLM が配置位置を点として出力し、その点への動作は別に生成します。本作の工夫は、配置位置を VLA の入力として与え、そこへ運んで放す動作まで VLA に学習させたことと、そのために学習データを追加せず、各デモの到達位置を事後的にラベルとして付けるだけで済ませたことです。

4. 結果

4.1 評価の方法

学習にも調整にも使っていない新しい 200 通りの初期状態で、すべての条件を同じ初期状態で回しました（1 試行は指示「全部片付けて」、全体で 200 s まで）。指標は、3 個とも箱に入った割合と、自力の立て直し率（掴み損ねのうち、方策が自分で掴み直して箱に入れた割合。1 回の試みでは最初の掴み損ねだけを数える）です。比べる条件と検定（McNemar 正確検定など、Holm 法で補正、$\alpha = 0.05$）は評価の前に決めて固定しました。検定の細部は README に書きました。

4.2 学び方ごとの結果

表 1　学び方ごとの結果（同じ 200 通りの初期状態）

学び方	学習データ	LLM の組み直し	3 個とも箱へ（95% 信頼区間）	自力の立て直し率（掴み損ねのうち）
成功データだけ	成功 240 本	なし	78（39.0%、32.5〜45.9%）	8/110（7.3%）
＋立て直しデータ	＋立て直し 90 本	なし	114（57.0%、50.1〜63.7%）	50/117（42.7%）
＋自分の失敗から学び直す	＋交代後 200 本・外乱 80 本・2 個入り 60 本	なし	144（72.0%、65.4〜77.8%）	59/71（83.1%）
＋LLM の組み直し	同上	あり	189（94.5%、90.4〜96.9%）	55/61（90.2%）
＋配置位置による目標条件付け	同上（配置位置を入力に加えて学び直す）	あり	193（96.5%、93.0〜98.3%）	67/71（94.4%）

上の 3 行が立て直しの学習の効果（LLM なし）、下の 2 行は学習とは別に重ねた仕組みです。自力の立て直し率の分母は掴み損ねの回数で、条件ごとに異なります。掴み損ねは事前に決めた定義（目標から 6 cm 以内でグリッパが閉じたのに、どの立方体も 1.5 cm 上がらない）で数えましたが、この定

義では、持ち上げて運んだ立方体を放すときにグリッパを一瞬開いて閉じ直した回も掴み損ねに入り、直後に箱に入ると立て直しにも数えられます。この回を除くと、上から 3.8%（4/106）、37.4%（40/107）、70.0%（28/40）、83.3%（25/30）、87.1%（27/31）で、表 2 の上の 2 つの比較はどちらも有意のままでした（README）。

表 2　事前に決めた比較

比較	指標	基準 → 比較対象	差（95% 信頼区間）	Holm 後の p
立て直しデータを足す	自力の立て直し率	7.3% → 42.7%	+35.5 pt（+23.9〜+47.1）	2.0×10^{-5}
自分の失敗から学び直す（LLM なし）	自力の立て直し率	42.7% → 83.1%	+40.4 pt（+26.2〜+54.0）	2.0×10^{-5}
自分の失敗から学び直す（LLM あり）	3 個とも箱へ	144 → 189	+22.5 pt（+15.4〜+29.5）	9.0×10^{-9}
配置位置による目標条件付け（600 通りの初期状態）	3 個とも箱へ	570 → 584	+2.3 pt（+0.2〜+4.5）	0.038（補正なし）

事前に決めた 3 つの比較は、Holm 法の補正の後も有意でした。最後の行は、失敗を分類した後に 3.5 節で足した比較で、1 本だけなので補正していません（$p = 0.038$）。失敗の分類に使った 600 通りは、この比較の 600 通りと同じです。表 1 の 200 通りを含む 600 通りの初期状態で、McNemar 正確検定（片方だけ成功した組は 13 と 27）で調べました。信頼区間の下限は 0 に近く、上がり幅は小さいと見ています。立て直しデータと LLM の組み直しの組み合わせ、外乱を入れた別の 100 通りの初期状態の結果は README に書きました。

4.3 追加の試み

- **自分の成功した試行で学び直す**: 3 個とも箱に入れた自分の試行 247 本を足して学び直しましたが、別の新しい 200 通りの初期状態で 145 → 142（McNemar $p = 0.81$）で差は確かめられず、採用していません。
- **候補から成功の見込みで選ぶ**: 方策が出す行動列の候補を 8 本作り、学習した成功の見込み（V-GPS（Nakamoto ら, 2024） と同じ考え方）が最も高いものを選びます。LLM なしでは別の新しい 200 通りの初期状態で 145 → 152（$p = 0.45$）、LLM ありでは 600 通りの初期状態で 570 → 578（$p = 0.30$）で、どちらも差は確かめられず、採用していません。

4.4 作品の変遷

図 4 に、モデルを改善してきた順序と、採用しなかった試みを示します。項目ごとに初期状態と回数が異なるので、数は項目の中だけで比べてください。図は開発した順で、LLM の組み直しは自分の失敗からの学び直しより先に作りました。表 1 は、同じ評価で学び方を積み上げた順に並べています。

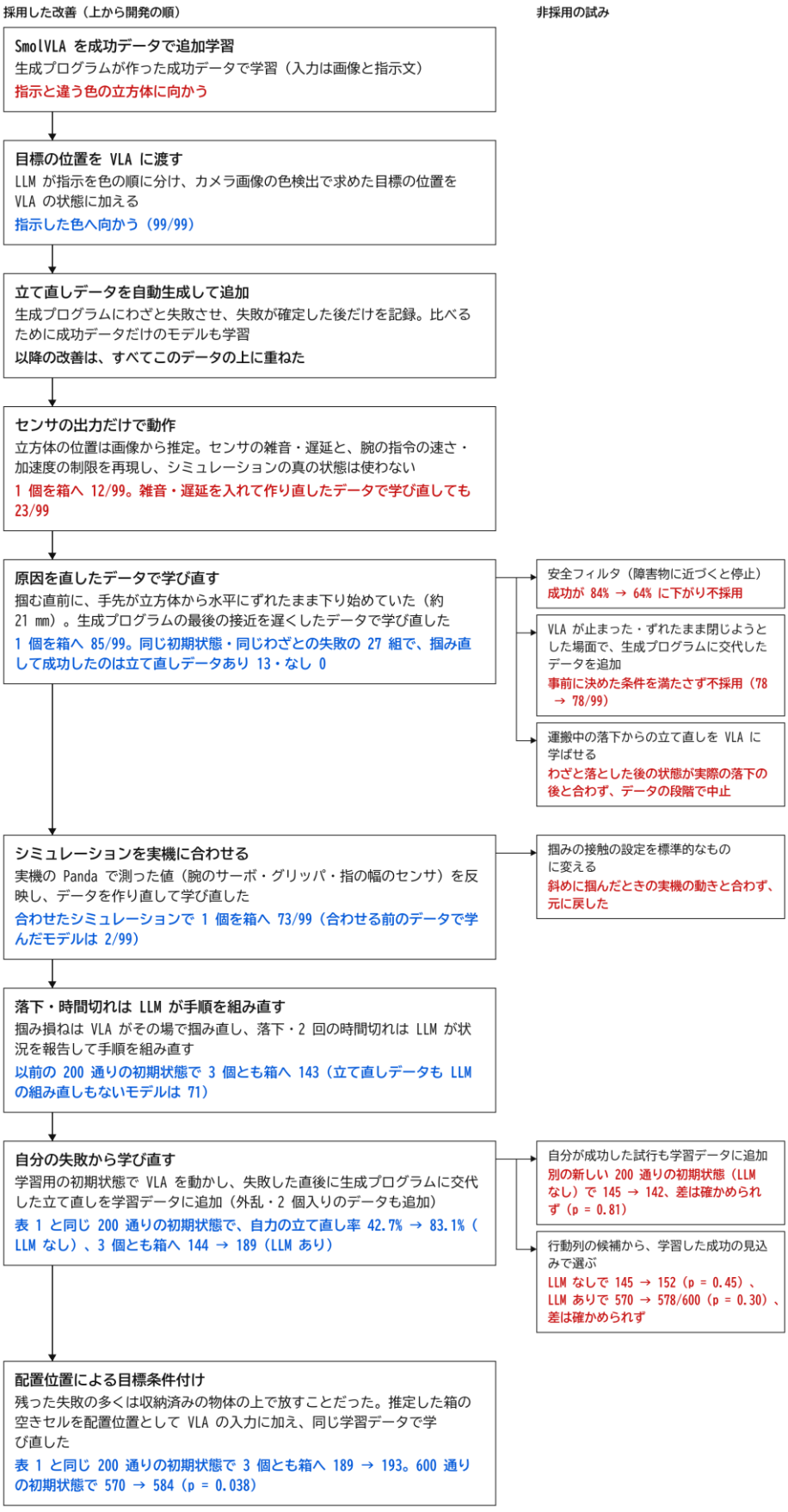


図 4　作品の変遷。左は採用した改善（上から開発の順）、右は非採用の試み。青は改善した結果、赤は課題・改善しなかった結果。/99 は開発中の 99 試行（立方体 1 個）、/200 は 200 通りの初期状態（3 個とも）。表 1 と同じ評価は最後の 2 項目だけ。

5. 考察

結果からは、立て直しを学ばせるには、どの失敗の直後のデータを使うかが重要であることがうかがえます。わざと起こした失敗からのデータで自力の立て直し率は 7.3% から 42.7% に、VLA 自身の失敗の直後のデータを足すと 83.1% に上がりました（運搬中の握り直しを除いても 3.8%、37.4%、70.0%）。わざと起こす失敗は生成プログラムが決めたずれに限られ、VLA が実際に起こす失敗とは状態の分布が異なるため、方策が実際に訪れる状態でデータを集めたことがこの差を埋めたと考えられます。自分の成功した試行を足しても成功が増えなかったこと（4.3 節）も、寄与したのが成功の量ではなく失敗の直後のデータであった可能性を示しています。方策だけでは抜け出せない落下や時間切れの繰り返しは LLM の組み直しが手順で補い、両方を合わせて 94.5% になりました。さらに、残った失敗を分類して最も多い種類だけを狙った配置位置による目標条件付けでは小さいながら上がり、狙いを決めずに足した候補選びでは差が確かめられなかったことから、失敗を数えてから次に直す所を決める進め方が改善につながったと見ています。

6. 今回の範囲と今後の課題（動画 2:52〜）

- 方策の評価はシミュレーションのみです。実機ではカメラがないため、シミュレーションで方策が出した腕の動きを再生しただけです。
- 同じ 200 通りの初期状態で条件を対にして比べる方法は、同じ初期状態と物理を何度でも再現できるシミュレーションだから成り立ちます。初期状態も接触も毎回わずかに変わる実機では、この比べ方は使えません。
- 学習のシード値は各モデル 1 つで、学習のばらつきは結果に含まれていません。
- 手先は真下向きに固定し、手首の向きを変えません。残った失敗の多くは、向きのずれた立方体での掴み損ねです。手首の回転を行動に加えることが次の課題です。
- 物を色で見分けています。指示の分解・知覚・完了の判定・VLA への指示のすべてが色を物の名前として使うので、同じ色の物が複数ある場面や、立方体以外の物が混ざった場面には広がりません。物に番号を振って分ける版の LLM は 20 の場面のうち 19 で正しく分けましたが（いまの版は 10。README）、動かすには知覚から VLA への指示まで変える必要があります。
- 自分の失敗からの学び直しと成功の見込みの学習は、このタスクの試行から行っており、タスクが変わるたびに試行を集め直す必要があります。多くのタスクに広げるには、タスクをまたいで使える成功の見込み（たとえば画像と言葉から成功を見積もるモデル）が要ります。
- 生成プログラムはシミュレーションの正確な位置を使います。形の決まった物（剛体）なら、産業用ロボットと同じく、推定した位置・姿勢から掴み方と経路を計算するスクリプト方策で、実機でも立て直しを含むデータを作れると考えています（本作で確かめたのは立方体だけです）。布や食品のような柔軟物は、状態を少数の数で表せず、失敗した後の形も毎回違うため、スクリプト方策ではデータを作りにくく、人の遠隔操作など別の手段が要ります。

7. 生成 AI の使用

本作の作成では、プログラムの作成と文章の推敲に生成 AI（Claude Code）を使いました。結果と動画は、実際の学習とシミュレーションの記録に基づきます。動画の読み上げには VOICEVOX（ずんだもん）を使いました。動画の映像は、評価と同じ設定で再実行した試行から、見せたい動作が起きたものを選び、倍速の場面は画面の右上に示しました。見出しの（）は動画の時刻です。

付録 A. 実時間で動かす手法 RTC を足したときの引き換え（動画 2:23〜）

推論の間も前の行動列の続きを実行する非同期実行では、新しい行動列に替わる所（つなぎ目）で動きに段差が生じます。RTC（Real-Time Chunking、Black ら, 2025）は、新しい行動列を前の行動列の残りに寄せて生成し、この段差を小さくする手法で、学習を変えずに推論のときだけ使えます。最終モデル（表 1 の最後の行）に RTC を足し、表 1 と同じ 200 通りの初期状態で比べました。寄せ方は 3 つで、原典の設定（44 ステップ＝4.4 秒先まで、重みを指数で減らす、上限 $\beta = 5$）、LeRobot の既定（10 ステップ先まで、線形、$\beta = 10$）、推論の間に実行する 4 ステップだけをそろえるもの（原典の hard masking）です。比べる条件と検定（3 個とも箱へは McNemar 正確検定、段差は Wilcoxon の符号順位検定、それぞれ 3 本を Holm 法で補正）は、評価の前に決めて固定しました。

表 A1　RTC の寄せ方ごとの結果（表 1 と同じ 200 通りの初期状態）

RTC の寄せ方	3 個とも箱へ	差（95% 信頼区間）	Holm 後の p	つなぎ目の段差（中央値）	掴み損ね	自力の立て直し率
なし（最終モデル）	193（96.5%）	—	—	5.5 mm/s	31 回	27/31（87.1%）
推論中の 4 ステップだけそろえる	198（99.0%）	+2.5 pt（−0.7〜+6.1）	0.36	4.5 mm/s	73 回	66/73（90.4%）
10 ステップ先まで寄せる（LeRobot の既定）	192（96.0%）	−0.5 pt（−4.7〜+3.6）	1	3.2 mm/s	154 回	123/154（79.9%）
44 ステップ先まで寄せる（原典の設定）	**158（79.0%）**	**−17.5 pt（−24.0〜−11.2）**	**5.3×10^{-7}**	3.4 mm/s	**329 回**	**149/329（45.3%）**

段差は、使う行動列が替わる所の手先の速度指令の差で、試行ごとの中央値をとりました。3 つの寄せ方とも段差は小さくなりました（Holm 後の p はいずれも 10^{-12} 未満）。一方、3 個とも箱に入った割合は、原典の設定だけが 96.5% から 79.0% に下がり（片方だけ成功した組は 41 と 6）、ほかの 2 つでは差は確かめられませんでした。掴み損ね（運搬中の握り直しを除き、1 回の試みで最初の 1 回）と自力の立て直し率は、判定に使わない記述です。

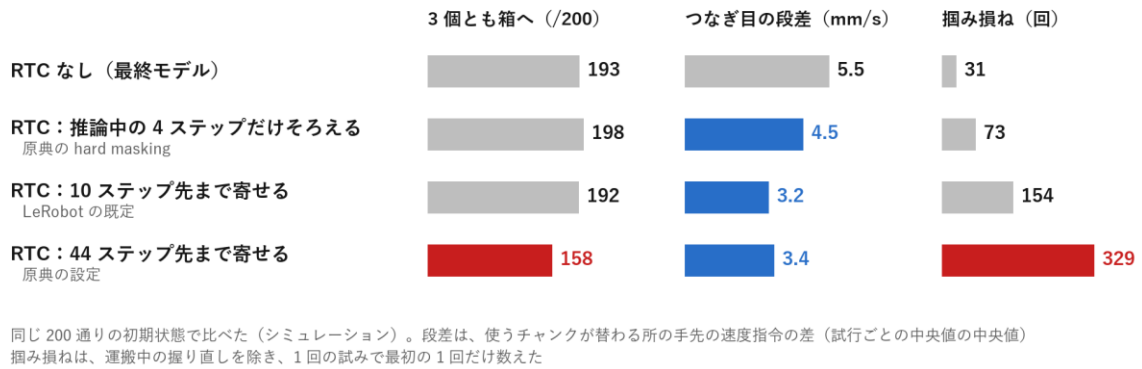


図 A1　RTC の寄せ方ごとの、3 個とも箱へ・つなぎ目の段差・掴み損ね（表 A1 の数）。青は改善、赤は悪化。

原典の設定で失敗した 42 本のうち 35 本は、目標の立方体を一度も持ち上げられないまま終わったものでした（RTC なしでは 7 本のうち 2 本）。掴み損ねた 1 秒後の手先の速度指令の向きを見ると、RTC なしでは 31 回すべてでほぼ止まっていたのに対し、原典の設定では 329 回のうち 63 回が箱の方（掴み損ねる前に運ぼうとしていた向き）へ、47 回が目標の立方体の方へ動いていました。新しい行動列を 4.4 秒先まで前の行動列に寄せるため、掴み損ねた後も、運ぶつもりで作った前の行動列に動きが引きずられたと考えています。掴み損ねそのものが増えた理由は、今回の記録からは切り分けていません。

動きを滑らかにする手法が、失敗からの立て直しと引き換えになりうることが分かりました。RTC が失敗からの立て直しに苦労するという記述は πR^2（Park・Tulsiani, 2026）にもありますが、実機の 20 試行の比較にとどまっていました。本作では、同じ 200 通りの初期状態の組で成功率の低下を検定で示し、掴み損ねの増加と、掴み損ねた後に箱の方へ動き続ける様子を記録で示しました。寄せる範囲を 10 ステップに絞れば成功率を保ったまま段差が小さくなったことから、寄せる範囲が滑らかさと立て直しの両立を分けると見ています。ただし、1 つのモデル・1 つのタスク・3 つの設定での観察です。範囲と強さを 1 つずつ変えた比較と、失敗を検知したときに寄せを外す方法の検証が次の課題です。